Ex.No:	
DATE:	

AIM:

ALGORITHM:

CODE:

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad
key = b'12345678'
cipher = DES.new(key, DES.MODE_ECB)
plaintext = input("Enter Plaintext: ")
ciphertext = cipher.encrypt(
  pad(plaintext.encode(), DES.block_size)
)
print("Encrypted Data:", ciphertext.hex())
decipher = DES.new(key, DES.MODE_ECB)
decrypted = unpad(
  decipher.decrypt(ciphertext),
  DES.block_size
)
print("Decrypted Data:", decrypted.decode())
```

OUTPUT:

```
Enter Plaintext: HELLO
Encrypted Data: 8b6a14412de58903
Decrypted Data: HELLO
```

RESULT:

Ex.No:	
DATE:	

AIM:

ALGORITHM:

CODE:

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
```

```
def mod_inverse(e, phi):
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d
p = 17
q = 11
n = p * q
phi = (p - 1) * (q - 1)
e = 7
d = mod_inverse(e, phi)
print("Public Key:", (e, n))
print("Private Key:", (d, n))
message = int(input("Enter Message (<187): "))
cipher = pow(message, e, n)
print("Encrypted Message:", cipher)
plain = pow(cipher, d, n)
print("Decrypted Message:", plain)
```

OUTPUT:

```
Public Key: (7, 187)
Private Key: (23, 187)
Enter Message (<187): 11
Encrypted Message: 88
Decrypted Message: 11
```

RESULT:

Ex.No:	
DATE:	

AIM:

ALGORITHM:

CODE:

```
import hashlib

message = input("Enter Message: ")

hash_value = hashlib.md5(message.encode())

print("MD5 Digest:", hash_value.hexdigest())
```

OUTPUT:

```
Enter Message: Cryptology
MD5 Digest: c6ae9d0a089311bab119969161ca5cce
```


RESULT:

Ex.No:	
DATE:	

AIM:

ALGORITHM:

CODE :

```
import hashlib
message = input("Enter Message: ")
 hash_value = hashlib.sha1(message.encode())
print("SHA-1 Digest:", hash_value.hexdigest())
```

OUTPUT :

```
Enter Message: Cryptology
SHA-1 Digest: ba7160026c971c61eca61f590d68b3de9fadc0f3
```

RESULT: